PLAYWRIGHT NOTES

1. What is Playwright?

- Playwright is an **open-source automation testing framework**.
- It is developed by **Microsoft**.
- It is mainly used for **automating and testing web applications**.
- It supports major browser engines:
    - **Chromium - chrome , edge**
    - **Firefox**
    - **WebKit - safari**
- It supports multiple programming languages such as:
    - JavaScript / TypeScript
    - Java
    - Python
    - .NET
- Playwright can be used for **end-to-end testing** of web applications.

Interview Definition

> Playwright is an open-source end-to-end test automation framework developed by Microsoft, used to automate and test web applications across Chromium, Firefox, and WebKit browsers.

---

2. Why Do We Use Playwright?

We use Playwright to:

- Automate web application testing.
- Reduce **manual testing effort**.
- Test applications across different browsers.
- Execute tests faster.
- Test complete **end-to-end user workflows**.
- Handle modern and dynamic web applications.
- Run multiple tests in parallel.
- Identify defects early.
- Improve test coverage.
- Reduce repetitive testing work.

> **Note : Flaky scripts : scripts might pass sometime and it might fail sometime(selenium cannot deal with dynamic elements(DOM Structure changes everytime))**

Interview Answer

> We use Playwright to automate web application testing, reduce manual effort, perform cross-browser testing, improve test coverage, and execute repetitive and end-to-end test scenarios efficiently.

---

3. Advantages of Playwright

1. Cross-Browser Support

- Supports **Chromium, Firefox, and WebKit**.
- The same test can be executed across different browsers.

2. Auto-Waiting → most import (30 sec)

- Playwright automatically waits for elements to become ready before performing actions.
- Reduces synchronization issues(speed of execution and speed of the web app) and flaky tests.

3. Fast Execution

- Playwright is designed for fast and efficient test execution.

4. Parallel Execution

- Multiple tests can run simultaneously.
- Reduces overall execution time.

5. Easy Browser Management

- Supports browser, browser context, and page management.
- Allows us to create isolated browser sessions.

6. Multiple Tab/Window Handling

- Can easily handle multiple tabs, windows, and popups.

7. Powerful Debugging

- Provides tools such as:
    - Playwright Inspector
    - Trace Viewer
    - Screenshots
    - Videos
    - HTML reports

8. Supports Modern Web Applications

- Works well with dynamic and modern web applications.

9. Open Source

- Playwright is freely available and has an active developer community.

---

4. Features of Playwright

Important features students should know:

- Cross-browser testing
- Auto-waiting
- Parallel test execution
- Multiple tab and window handling
- Iframe/Frame handling
- Popup handling
- Dialog handling
- Mouse and keyboard actions
- File upload and download
- Screenshots and video recordings
- Assertions
- Hooks
- Annotations
- Test retries
- Test timeout management - 30 sec
- Trace Viewer
- HTML reporting

---

5. Playwright Test Runner → execution takes place

Playwright provides its own **built-in test runner**, called **Playwright Test**.

It provides features such as:

- Test execution
- Assertions
- Fixtures
- Hooks
- Parallel execution
- Annotations
- Reporting

Interview Point

> Playwright Test is the built-in test runner provided by Playwright for creating, executing, managing, and reporting automated tests.

---

6. Playwright – Quick Interview Revision

What is Playwright?

> An open-source end-to-end test automation framework developed by Microsoft for testing web applications, api testing, mobile automation.

Why do we use Playwright?

> To automate web application testing, reduce manual effort, perform cross-browser testing, and execute tests efficiently.

Which browsers does Playwright support?

> Chromium, Firefox, and WebKit.

Which languages does Playwright support?

> JavaScript, TypeScript, Python, Java, and .NET.

Is Playwright open source?

> Yes, Playwright is an open-source framework developed by Microsoft.

What is Playwright's biggest advantage?

> It provides reliable cross-browser automation with features such as auto-waiting, parallel execution, browser contexts, powerful debugging.

Installation:

1. **Install Node.js** – Playwright requires Node.js and npm.

   Why required? Playwright's JavaScript/TypeScript tooling runs on **Node.js**.
   Use: Provides the **runtime environment** to execute Playwright tests.

   What is it? npm = **Node Package Manager**.
   Why required? Used to **install and manage Playwright and other project dependencies**.
   Example: `npm init playwright@latest`

2. **Install Visual Studio** – For writing test scripts
3. **Check installation** – Verify using `node -v` and `npm -v`.
4. **Create project folder** – Create and open the project in VS Code.
5. **Open Terminal** – Open the terminal inside the project folder.
6. **Initialize Playwright** – Run `npm init playwright@latest`.
   (if any error: npm.cmd init playwright@latest)
7. **Select language** – Choose **JavaScript** or TypeScript.

Playwright folder structure:

```
Playwright Project
│
├── tests/
│   └── example.spec.js
│
├── playwright.config.js
├── package.json
├── package-lock.json
├── node_modules/
```

1. `tests/`

- Main folder where we **store our test cases**.
- Usually contains files ending with `.spec.js` or `.test.js`.

Example: `login.spec.js`

2. `playwright.config.js`

- The **main configuration file** of Playwright.
- Used to configure things like:
    - Browsers
    - Base URL
    - Timeout
    - Retries
    - Workers
    - Reports
    - Screenshots/videos
    - Projects

---

3. `package.json`

- Contains the **project information and dependencies**.
- Stores Playwright and other npm package details.
- Also contains scripts used to run the project.

---

4. `package-lock.json`

- Keeps the **exact versions of installed npm packages and their dependencies**.
- Helps maintain consistent installations across environments.

---

5. `node_modules/`

- Contains the **installed packages/dependencies** required by the project.
- Created automatically when npm installs packages.

Important: We generally don't manually modify this folder.

Interview Point:

> The Playwright project structure mainly consists of the tests folder for test cases, playwright.config.js for configuration, package.json for dependencies and project scripts, node_modules for installed packages, and report/result folders for test execution artifacts.

First code :

```
import {test} from "@playwright/test"

test("first code" , ()=>{

    console.log("hello");

})
```

Fixtures:

Fixtures are configurations or environments provided by Playwright that provide the required resources for executing a test.

Types of fixtures:

1. **Browser fixture :** Browser fixture is a pre-configured environment provided by Playwright that gives us a browser instance to run our tests.

2. **Context fixture :** Context fixture is a pre-configured environment provided by Playwright that gives us a browser context to run tests in an isolated session, where incognito mode is the default.

3. **Page fixture :** Page fixture is a pre-configured environment provided by Playwright that gives us a tab (page) to interact with and perform actions on a web application.

4. **browserName fixture :** Browser name fixture provides the name of the browser in which the test is being executed, such as Chromium, Firefox, or WebKit.

5. **Request :** This fixture is used in API automation

```
import {test} from "@playwright/test"



//browser fixture

test("fixtures", async({browser})=>{  //tells the browser

    let context = await browser.newContext()  //browser session(isolated
session(profile), incognito mode)

    let page = await context.newPage()  //tab within the broswer session

    await page.goto("https://www.amazon.in/")

}) //control to multiple conext(browser session) mutiple windows
```

```
//context

test ("context fixture", async({context})=>{   //default it will create browser session
in the all the 3 browser

    let page = await context.newPage()

    await page.goto("https://www.amazon.in/")

}) //control over multiple tabs within one session



//page

test("page fixture", async({page})=>{    //tab inside the default 3 browsers

    await page.goto("https://www.amazon.in/")

}) //control over a single tab



//browsername

test("browsername fixture", async({browserName})=>{

    console.log(browserName);

})//return the name of the browser which is in used
```

Execute the test file in headed mode:

npx playwright test tests/example.spec.js --headed

Execute the test file in headless mode:

npx playwright test tests/example.spec.js